

UNIVERSITY OF MORATUWA

Faculty of Engineering

Department of Electronic and Telecommunication Engineering

EN4554

Assignment 2

Convolutional Neural Networks on CIFAR-10

Name : Eashan S.G.S

Index No. : 220148G

Overview

GitHub repository: `https://github.com/sahas-eashan/EN4554-Assignment-2-CNNs`
The repository holds the full Jupyter notebook (`220148G_a02.ipynb`) with every cell output. This report shows the important parts of the code, the results and their interpretation.

Setup.

- **Framework:** PyTorch, run on an NVIDIA RTX 3090.
- **Data split:** the CIFAR-10 training set is split into **45k train / 5k validation** with a fixed seed.
- **Normalization:** per channel, using the statistics of the 45k training split only: mean $(0.491, 0.482, 0.446)$, standard deviation $(0.247, 0.244, 0.262)$.
- **Data pipeline:** the whole dataset is kept on the GPU, and mini-batching and all augmentations run there.
- **Test set:** used **once**, in Task 4.
- **Reproducibility:** all random seeds are fixed.

Use of AI tools. Claude (Anthropic) was used as a coding assistant to write and debug the code and to help run the experiments on the lab GPU server. I have reviewed and understood all of the code and the results.

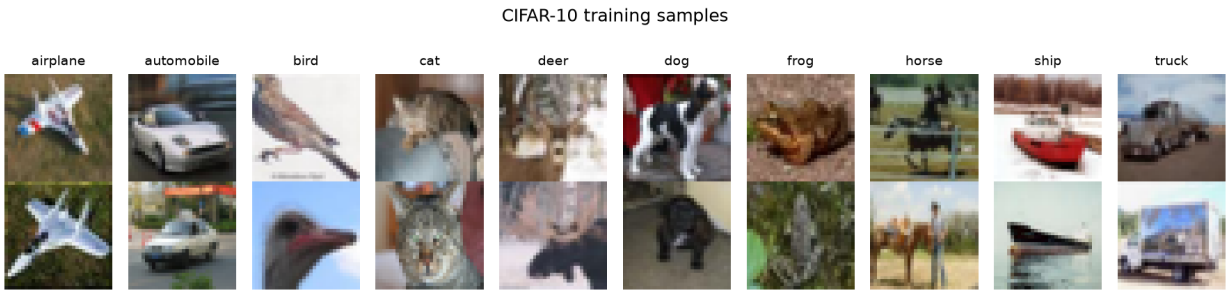


Figure 1: Two training images from each CIFAR-10 class.

1. Backpropagation

1(a) Layers with explicit forward/backward passes

Each layer caches what its backward pass needs. For an upstream gradient $\delta = \partial \ell / \partial \,\text{out}$:

Linear $y = xW + b$:	$\frac{\partial \ell}{\partial W} = x^\top \delta, \quad \frac{\partial \ell}{\partial b} = \sum_n \delta_n, \quad \frac{\partial \ell}{\partial x} = \delta W^\top$
ReLU :	$\frac{\partial \ell}{\partial x} = \delta \odot \mathbf{1}[x > 0]$
Sigmoid $s = \sigma(x)$:	$\frac{\partial \ell}{\partial x} = \delta \odot s(1 - s)$
Softmax CE $\ell = -\frac{1}{N}\sum_n \log p_{n,y_n}$:	$\frac{\partial \ell}{\partial z} = \frac{1}{N}(p - \text{onehot}(y))$

No autograd is used: none of the tensors require gradients.

```
class Linear:
    def __init__(self, n_in, n_out, init="he", dtype=torch.float32, device=DEVICE):
        # He init for ReLU, Xavier init for sigmoid
        std = math.sqrt(2.0 / n_in) if init == "he" else math.sqrt(2.0 / (n_in + n_out))
        self.W = torch.randn(n_in, n_out, dtype=dtype, device=device) * std
```

```
        self.b = torch.zeros(n_out, dtype=dtype, device=device)
    def forward(self, x):
        self.x = x
        return x @ self.W + self.b
    def backward(self, dout):
        self.dW = self.x.T @ dout
        self.db = dout.sum(0)
        return dout @ self.W.T

class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask
    def backward(self, dout):
        return dout * self.mask

class SoftmaxCrossEntropy:
    def forward(self, z, y):
        z = z - z.max(dim=1, keepdim=True).values             # numerical stability
        logp = z - torch.logsumexp(z, dim=1, keepdim=True)
        self.p, self.y = logp.exp(), y
        return -logp[torch.arange(len(y)), y].mean()
    def backward(self):
        g = self.p.clone()
        g[torch.arange(len(self.y)), self.y] -= 1
        return g / len(self.y)
```

The `MLP` class stacks `Linear` and activation layers. Its `backward()` walks the layers in reverse, passing each layer's input gradient to the layer below.

1(b) Gradient check

A small MLP (20–16–12–10, ReLU) was checked in float64 on a random batch of 8 samples. The manual gradients were compared against:

1. **autograd**, using the same weights in an equivalent PyTorch network;
2. **centred finite differences**, $\frac{\ell(\theta_i+\varepsilon)-\ell(\theta_i-\varepsilon)}{2\varepsilon}$ with $\varepsilon = 10^{-6}$, for *every* parameter.

The relative error is $\|g_1 - g_2\|_2 / (\|g_1\|_2 + \|g_2\|_2)$.

```
def finite_diff(net, x, y, eps=1e-6):
    out = []
    for p in net.params():
        flat, g = p.view(-1), torch.zeros(p.numel(), dtype=p.dtype)
        for i in range(flat.numel()):
            old = flat[i].item()
            flat[i] = old + eps; lp = net.loss(x, y).item()
            flat[i] = old - eps; lm = net.loss(x, y).item()
            flat[i] = old
            g[i] = (lp - lm) / (2 * eps)
        out.append(g.view_as(p))
    return out
```

Table 1: Gradient check (loss: manual 3.757791109124, autograd 3.757791109124).

parameter	shape	rel. err vs autograd	`allclose`	rel. err vs finite diff.	`allclose`
W_1	20×16	4.57×10^{-17}	True	6.13×10^{-10}	True
b_1	16	5.98×10^{-17}	True	5.00×10^{-10}	True
W_2	16×12	1.94×10^{-17}	True	3.99×10^{-10}	True
b_2	12	2.54×10^{-17}	True	3.51×10^{-10}	True
W_3	12×10	1.31×10^{-17}	True	4.40×10^{-10}	True
b_3	10	2.31×10^{-17}	True	1.00×10^{-9}	True

Interpretation.

- **Against autograd:** the manual gradients agree to machine precision (about 10^{-17}), the two losses are identical to 12 digits, and `torch.allclose` passes for every parameter.
- **Against finite differences:** the errors of 10^{-10}–10^{-9} are exactly the expected $O(\varepsilon^2)$ truncation error plus round-off.
- The finite-difference check uses no framework, so it validates the derivations themselves, not just consistency with PyTorch.

1(c) Training the MLP with SGD

The from-scratch MLP is 3072–512–256–10 with ReLU and He initialization. It is trained on flattened images with plain mini-batch SGD (batch 128, learning rate 0.02, 20 epochs), using a hand-written update:

```
for xb, yb in batches(Xf_tr, Ytr, bs):
    loss = net.loss(xb, yb)
    net.backward()
    for p, g in zip(net.params(), net.grads()):
        p -= lr * g                          # plain SGD step (in place)
```

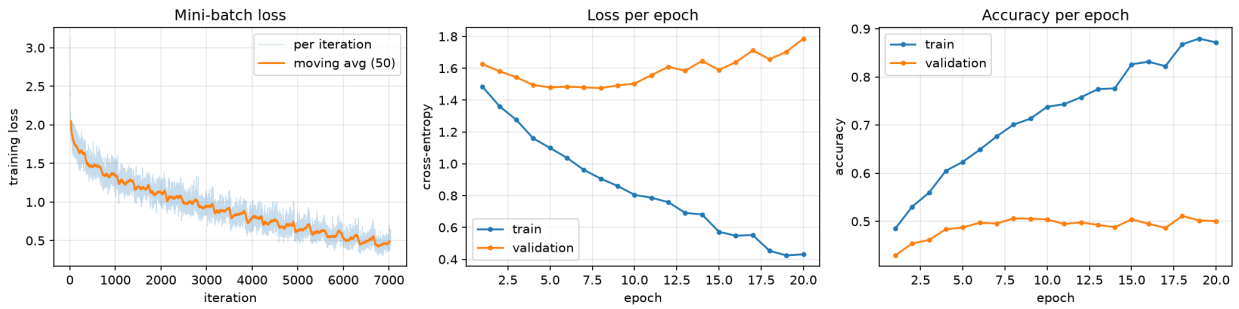


Figure 2: From-scratch MLP: mini-batch loss, per-epoch loss and per-epoch accuracy.

Result: best validation accuracy **51.1 %** (epoch 18); final validation accuracy **50.1 %**.

Interpretation.

- **Loss curve:** the mini-batch loss drops quickly from 2.3 to about 1.5 in the first epoch, then falls steadily to about 0.5. The noise around the moving average comes from SGD sampling.
- **Accuracy:** about 50 % is typical for a fully connected network on raw pixels. Flattening discards spatial structure, and without weight sharing every pattern must be learned separately

at every position.

- **Overfitting:**
  - The validation loss reaches its minimum (about 1.47) around epochs 5–8 and then rises to 1.78.
  - Over the same period the training accuracy climbs to 87 % while validation accuracy stays flat near 50 %.
  - The 1.7M-parameter MLP is memorizing the training set, so early stopping or regularization would be needed.

1(d) Gradient norms at initialization: sigmoid vs ReLU

The test network is a 10-layer MLP: 3072 inputs, 9 hidden layers of width 256, and 10 outputs.

- The sigmoid version uses Xavier initialization and the ReLU version uses He initialization, i.e. each activation gets its standard initialization.
- One forward/backward pass on 512 training images gives $\|\partial \ell / \partial W_l\|_F$ for every layer.

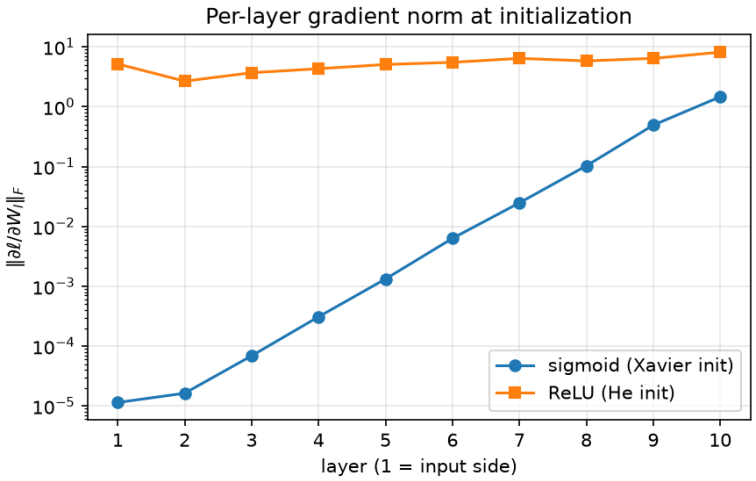


Figure 3: Per-layer weight-gradient norm at initialization (log scale).

Table 2: $\|\partial \ell / \partial W_l\|_F$ per layer (layer 1 = input side).

layer	1	2	3	4	5	6	7	8	9	10
sigmoid	1.2e-5	1.6e-5	6.9e-5	3.1e-4	1.3e-3	6.3e-3	2.5e-2	0.10	0.49	1.46
ReLU	5.16	2.66	3.69	4.30	5.06	5.49	6.40	5.80	6.40	8.11

Interpretation.

- **Sigmoid:** the gradient norm falls by about **five orders of magnitude** from the output layer to the input layer (layer-1/layer-10 ratio 7.8×10^{-6}). This is the **vanishing-gradient** problem.
  - Each backward step multiplies the gradient by $\sigma'(z) \leq 0.25$, and by $W^\top$, whose gain under Xavier initialization is about 1.
  - The gradient therefore shrinks by roughly $4\times$ per layer, which is the straight line on the log plot, and the early layers barely learn.

- **ReLU:** the norms stay within 2.7–8.1 across all layers (ratio 0.64).
  - $\mathrm{ReLU}' \in \{0, 1\}$, so about half the units pass the gradient through unchanged.
  - He initialization ($\mathrm{Var}(W) = 2/n_{\text{in}}$) compensates exactly for that factor $\frac{1}{2}$, so the gradient variance is preserved with depth.

2. Optimizers

2(a) SGD with momentum and Adam as `torch.optim.Optimizer` subclasses

For a gradient g_t (with L2 weight decay, $g_t \leftarrow g_t + \lambda\theta$):

- **SGD with momentum:** $b_t = \mu b_{t-1} + g_t$, $\theta \leftarrow \theta - \eta b_t$, with $b_1 = g_1$.
- **Adam:** $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$. The bias-corrected moments are $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$, and the update is $\theta \leftarrow \theta - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$.

```python
class MySGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-2, momentum=0.0, weight_decay=0.0):
        super().__init__(params, dict(lr=lr, momentum=momentum, weight_decay=weight_decay))
    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            lr, mu, wd = group["lr"], group["momentum"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad if wd == 0 else p.grad.add(p, alpha=wd)
                if mu != 0:
                    state = self.state[p]
                    if "buf" not in state:
                        state["buf"] = g.clone()
                    else:
                        state["buf"].mul_(mu).add_(g)
                    g = state["buf"]
                p.add_(g, alpha=-lr)

class MyAdam(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.0):
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps, weight_decay=weight_decay))
    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            lr, (b1, b2) = group["lr"], group["betas"]
            eps, wd = group["eps"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad if wd == 0 else p.grad.add(p, alpha=wd)
                state = self.state[p]
                if not state:
                    state["t"] = 0
                    state["m"] = torch.zeros_like(p)
                    state["v"] = torch.zeros_like(p)
                state["t"] += 1
                t, m, v = state["t"], state["m"], state["v"]
                m.mul_(b1).add_(g, alpha=1 - b1)
                v.mul_(b2).addcmul_(g, g, value=1 - b2)
```

```
            m_hat, v_hat = m / (1 - b1 ** t), v / (1 - b2 ** t)
            p.addcdiv_(m_hat, v_hat.sqrt().add_(eps), value=-lr)
```

(The closure handling is omitted here; it is in the notebook.)

Equivalence test. Two copies of the same float64 network are trained side by side for 200 steps on identical mini-batches, one with my optimizer and one with the `torch.optim` reference.

Table 3: Custom vs. PyTorch optimizers after 200 steps.

configuration	max $\lvert\theta_{\text{mine}} - \theta_{\text{torch}}\rvert$	`torch.allclose`
SGD ($\eta = 0.1$)	0	True
SGD momentum ($\eta = 0.05$, $\mu = 0.9$, wd 10^{-3})	0	True
Adam ($\eta = 10^{-2}$)	4.4×10^{-16}	True
Adam ($\eta = 10^{-3}$, $\beta = (0.8, 0.99)$, wd 10^{-3})	8.3×10^{-17}	True

Interpretation.

- The SGD variants match PyTorch **bit-for-bit**.
- Adam differs only by round-off. PyTorch folds the bias corrections into the step size and the denominator, which is algebraically identical to the textbook form used here.

2(b) Trajectories on $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$

The Hessian is $\text{diag}(1, 50)$, so the condition number is $\kappa = 50$. All three methods start from $(-4.5, 1)$ and run for 100 steps:

- SGD and SGD with momentum ($\mu = 0.9$) both use $\eta = 0.03$. Gradient descent is stable only for $\eta < 2/50$.
- Adam uses $\eta = 0.3$.

The gradient is exact, so "SGD" here is plain gradient descent.

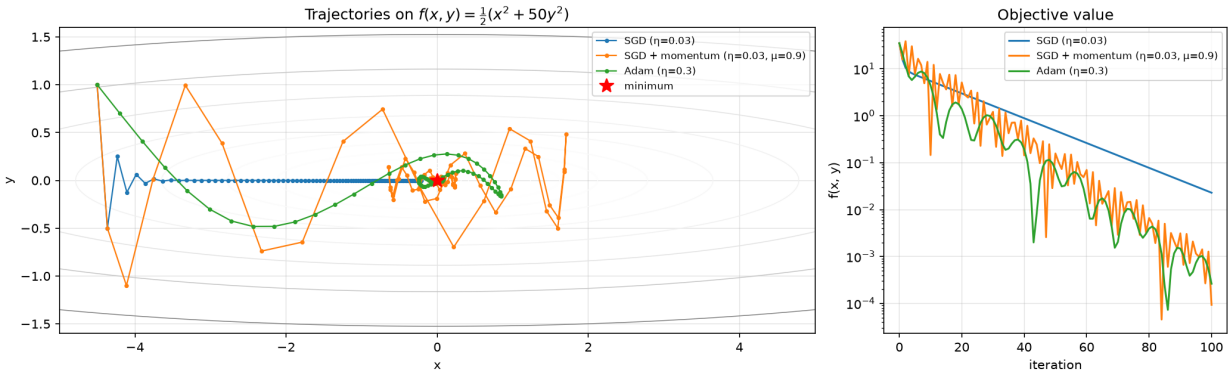


Figure 4: Optimizer trajectories over the contours of f (left) and the objective value per iteration (right).

Table 4: Position and objective after 100 steps.

optimizer	final (x, y)	$f(x, y)$
SGD ($\eta = 0.03$)	$(-0.214,\ 0.000)$	2.29×10^{-2}
SGD + momentum ($\eta = 0.03$, $\mu = 0.9$)	$(0.011,\ -0.001)$	9.34×10^{-5}
Adam ($\eta = 0.3$)	$(-0.020,\ -0.002)$	2.63×10^{-4}

Interpretation.

- **SGD:**
  - Along the steep y direction each step multiplies the error by $1 - 50\eta = -0.5$. The first steps zig-zag across the valley, and y then dies out quickly.
  - Along the flat x direction the factor is only 0.97, so the iterate creeps forward and is still at $x = -0.21$ after 100 steps.
  - A learning rate that is safe for the steep direction is tiny for the flat one. This is the ill-conditioning problem.
- **SGD + momentum:**
  - The velocity accumulates the consistent x-gradient, giving roughly $1/(1 - \mu) = 10\times$ faster progress along the valley. The final f is $250\times$ lower than SGD's.
  - With $\mu = 0.9$ the system is strongly under-damped. The path swings far past the minimum (out to $x \approx 1.7$), and f oscillates instead of decreasing monotonically.
- **Adam:**
  - Each coordinate is normalized by its own gradient RMS, so the first steps have size about η in *both* directions, a diagonal move along $-\text{sign}(\nabla f)$. The steep y direction no longer dictates the step.
  - The path is smooth. It overshoots to $x \approx 0.85$ and then spirals in.
- Momentum and adaptive scaling are two different remedies for ill-conditioning. Both beat gradient descent by about two orders of magnitude.

2(c) Optimizer comparison on a small CNN

Model: conv32–conv32–pool–conv64–conv64–pool–conv128–pool–FC256–FC10 (666,538 parameters). It has **no batch norm**, so normalization does not mask the optimizer's behaviour.

Training: each optimizer from 2(a) is trained at $\eta \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ for 10 epochs on the 45k training split, with batch size 128, a constant learning rate and no augmentation.

Table 5: Validation accuracy after 10 epochs (best per optimizer in bold).

optimizer	$\eta = 10^{-3}$	$\eta = 10^{-2}$	$\eta = 10^{-1}$
SGD	13.6 %	43.8 %	**68.0 %**
SGD + momentum	47.6 %	**74.9 %**	55.2 %
Adam	**75.1 %**	52.4 %	10.0 %

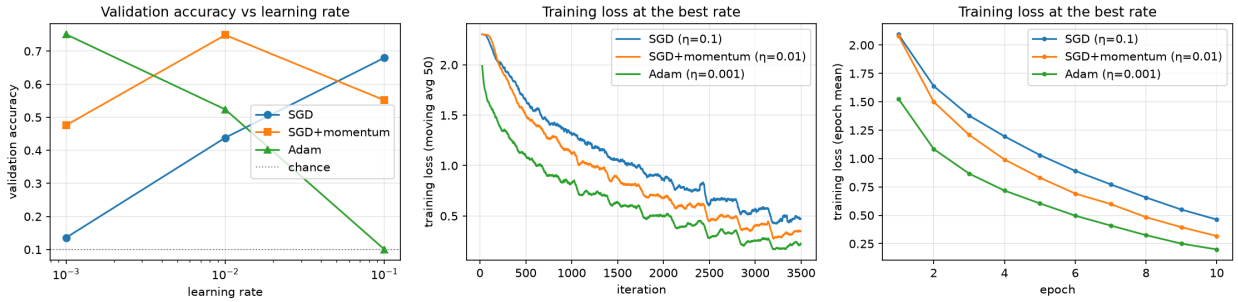


Figure 5: Validation accuracy versus learning rate (left), and training loss at each optimizer's best rate (middle and right).

Interpretation.

- **Each optimizer has its own sweet spot,** one decade lower for each step from SGD to momentum to Adam:
  - Momentum's effective step is about $\eta/(1-\mu) = 10\eta$, so it wants a 10× smaller rate than SGD.
  - Adam normalizes every update to about η per parameter, so it needs the smallest rate.
- **High sensitivity:** one decade away from the best rate costs 25–65 points.
  - SGD at 10^{-3} barely leaves chance level (13.6 %).
  - Adam at 10^{-1} never learns: its loss stays at $\ln 10 \approx 2.30$ (chance), as the huge per-parameter steps immediately drive the ReLUs into a dead or saturated state.
  - SGD's best rate is at the edge of the grid, so it would probably improve further at a higher rate.
- **Best results:** at their best rates, Adam and momentum both reach about **75 %**, ahead of plain SGD (68 %).
- **Training speed:** Adam reduces the training loss fastest throughout, followed by momentum and then plain SGD. Faster convergence only holds when the learning rate is tuned per optimizer.

3. Regularization and Residual Connections

3(a) Regularization on 5k training images

Setup:

- **Data:** a class-balanced subset of 5,000 training images (500 per class). Evaluation uses the 5k validation split.
- **Model:** the SmallCNN from 2(c) **with batch norm**.
- **Training:** SGD with momentum 0.9, learning rate 0.05 with cosine decay, batch size 128, 60 epochs.
- **Settings compared:**
  - weight decay 10^{-3};
  - dropout $p = 0.5$ on the FC layers;

  - random flip + 4-pixel-padded crop;
  - cutout (one 16×16 hole);
  - mixup ($\alpha = 0.4$);
  - all of these combined.

- **Train accuracy** is measured on the clean training images in eval mode, so it is comparable across settings.

Cutout and mixup are my own implementations, applied per batch on the GPU:

```
def cutout(x, size=16):
    B, C, H, W = x.shape
    cy = torch.randint(0, H, (B, 1, 1), device=x.device)
    cx = torch.randint(0, W, (B, 1, 1), device=x.device)
    yy = torch.arange(H, device=x.device)[None, :, None]
    xx = torch.arange(W, device=x.device)[None, None, :]
    hole = ((yy >= cy - size // 2) & (yy < cy + size // 2) &
            (xx >= cx - size // 2) & (xx < cx + size // 2))
    return x * (~hole)[:, None].to(x.dtype)

def mixup(x, y, alpha):
    lam = float(np.random.beta(alpha, alpha))
    j = torch.randperm(len(x), device=x.device)
    return lam * x + (1 - lam) * x[j], y, y[j], lam
# loss = lam * CE(out, y) + (1 - lam) * CE(out, y[j])
```

Table 6: Regularization results on 5k training images (60 epochs).

setting	train acc	val acc	gap	best val acc
baseline	100.0 %	66.5 %	33.5	66.5 %
weight decay	100.0 %	66.9 %	33.1	67.2 %
dropout	93.9 %	67.3 %	26.6	67.7 %
flip+crop	93.3 %	**74.7 %**	18.6	**75.0 %**
cutout	99.9 %	70.1 %	29.8	70.3 %
mixup	99.9 %	67.4 %	32.5	67.7 %
all combined	67.3 %	62.8 %	**4.5**	63.4 %

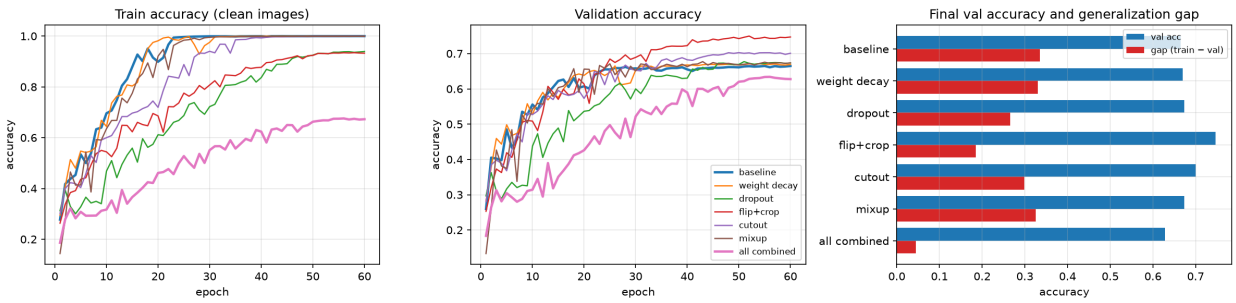


Figure 6: Training accuracy (clean images) and validation accuracy per epoch, and the final validation accuracy and generalization gap.

Interpretation.

- **Baseline:** the BN-CNN reaches 100 % training accuracy by about epoch 25, while validation accuracy plateaus at 66.5 %. A 33-point gap means the network has memorized the 5k images.
- **Weight decay:** almost no effect (+0.4 points). With batch norm, the scale of the conv weights does not change the function, so L2 mainly alters the effective learning rate rather than limiting capacity.
- **Dropout:** slows memorization (training accuracy 94 %) and shrinks the gap to 27 points, but adds only 0.8 points of validation accuracy.
- **Flip+crop:** by far the best single method (**+8.2 points**, gap 19). Its label-preserving variants match real invariances of natural images, which effectively enlarges the dataset.
- **Cutout:** also helps (+3.6 points).
- **Mixup:** with $\alpha = 0.4$ it barely regularizes here (+0.9 points); clean training accuracy is still 99.9 %.
- **All combined:** gives the smallest gap (4.5 points) but the *lowest* validation accuracy, 62.8 %. The model **underfits**: training accuracy is only 67 % and still rising at epoch 60.
  - Regularization trades variance for bias, and stacking everything would need a longer schedule or weaker settings.
  - A small gap alone does not mean good generalization.

3(b) Plain vs. residual networks of depth $6n + 2$

Architecture (CIFAR ResNet of He et al.):

- a 3×3 conv stem with 16 channels;
- three stages of n basic blocks with 16/32/64 channels, where each block is conv–BN–ReLU–conv–BN (+ shortcut) followed by ReLU;
- global average pooling and an FC layer;
- Kaiming initialization;
- depths 8, 20 and 56 ($n = 1, 3, 9$).

The **only difference between the two families is the identity shortcut**. Where the shape changes, it subsamples and zero-pads the channels (option A), so it adds no parameters.

Training (identical for all six networks):

- flip+crop augmentation;
- SGD with momentum 0.9 and learning rate 0.1, with a 2-epoch linear warm-up then cosine decay;
- weight decay 10^{-4}, batch size 128;
- 30 epochs with mixed precision.

```
class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride, residual):
        super().__init__()
        self.conv1, self.bn1 = conv3x3(cin, cout, stride), nn.BatchNorm2d(cout)
        self.conv2, self.bn2 = conv3x3(cout, cout), nn.BatchNorm2d(cout)
        self.residual, self.stride, self.extra = residual, stride, cout - cin
    def shortcut(self, x):   # option A: parameter-free identity / subsample + zero-pad channels
        if self.stride != 1:
```

```
            x = x[:, :, ::self.stride, ::self.stride]
        if self.extra:
            x = F.pad(x, (0, 0, 0, 0, self.extra // 2, self.extra - self.extra // 2))
        return x
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:                # the ONLY difference between plain and residual
            out = out + self.shortcut(x)
        return F.relu(out)
```

Table 7: Plain vs. residual networks after 30 epochs (identical parameter counts per depth).

network	parameters	final train loss	final train acc	final val acc
plain-8	75,290	0.381	87.0 %	84.8 %
residual-8	75,290	0.400	86.3 %	84.5 %
plain-20	269,722	0.224	92.4 %	87.5 %
residual-20	269,722	0.168	94.2 %	89.0 %
plain-56	853,018	**0.685**	76.1 %	75.7 %
residual-56	853,018	**0.090**	97.0 %	**89.9 %**

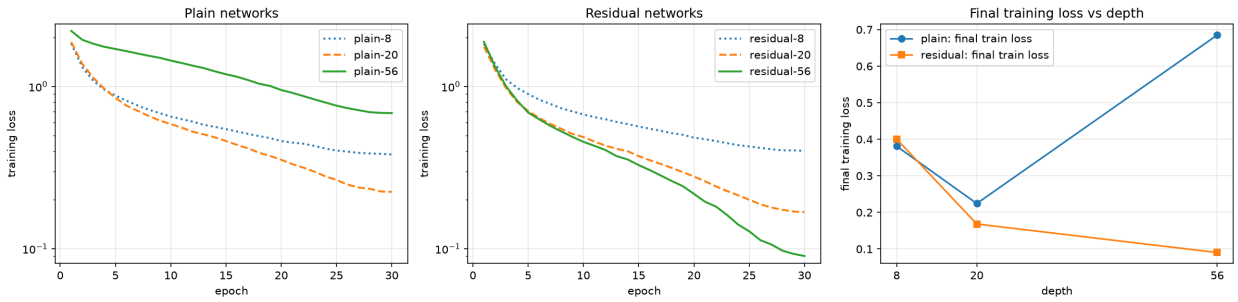


Figure 7: Training loss per epoch for the plain and residual families, and the final training loss against depth.

Interpretation.

- **Plain networks show the degradation problem.**
  - Going from 8 to 20 layers helps, but plain-56 is far worse than plain-20: its training loss is 3× higher, and its curve lies above plain-20's from the first epoch.
  - The deeper network is worse on the *training* set, so this is an **optimization failure, not overfitting**. A 56-layer network could in principle copy the 20-layer solution with identity layers, but SGD does not find it.
  - In a first attempt without warm-up, plain-56 even diverged to NaN in epoch 1, while the residual networks trained normally.
- **Residual networks: deeper is better.** The final training loss falls monotonically with depth (0.400, 0.168, 0.090), and residual-56 is the best network (89.9 %).
- **Why the shortcut helps:**
  - Each block only learns a residual $F(x)$ on top of x, so the identity is the easy default.

  - The block Jacobian $I + \partial F/\partial x$ gives gradients a direct path to the early layers, instead of a product of 54 conv–BN–ReLU stages.

- **Depth dependence:** at depth 8 the two families are practically identical, and the benefit of the shortcut grows with depth. Parameter counts are identical, so every difference comes from the shortcut alone.

4. Best CNN with at most 500k Parameters and 20 Epochs

Design: a compact ResNet-14.

- a 3×3 stem with 32 channels;
- three stages of **2 basic blocks** with **32/64/96 channels**, stride 2 at the start of stages 2 and 3;
- 1×1 projection shortcuts where the shape changes;
- global average pooling and an FC layer;
- **481,962 parameters** in total.

Full training recipe:

- 20 epochs, batch size 128;
- SGD with Nesterov momentum 0.9 and a one-cycle learning-rate schedule (peak 0.1, 25 % warm-up);
- weight decay 5×10^{-4};
- label smoothing 0.1;
- augmentation: flip+crop + cutout;
- mixed precision.

Tuning: only on the validation split. The test set is evaluated **once**, for the configuration selected on validation.

```
class Block4(nn.Module):
    def __init__(self, cin, cout, stride, residual=True):
        super().__init__()
        self.conv1, self.bn1 = conv3x3(cin, cout, stride), nn.BatchNorm2d(cout)
        self.conv2, self.bn2 = conv3x3(cout, cout), nn.BatchNorm2d(cout)
        self.residual, self.proj = residual, None
        if residual and (stride != 1 or cin != cout):
            self.proj = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False),
                                      nn.BatchNorm2d(cout))
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:
            out = out + (x if self.proj is None else self.proj(x))
        return F.relu(out)

opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9, nesterov=True,
                      weight_decay=5e-4)
h = train(model, opt, 20, Xtr, Ytr, Xva, Yva, bs=128, aug=compose(flip_crop, cutout),
          sched="onecycle", amp=True, label_smoothing=0.1)
```

Ablation. Each variant changes exactly one design choice of the full recipe.

Table 8: Ablation (final validation accuracy after 20 epochs). Full recipe: 89.80 %.

variant	change	val acc	Δ vs. full
no augmentation	no flip, crop or cutout	86.22 %	−3.58
no weight decay	$\lambda = 0$	88.12 %	−1.68
AdamW	AdamW (peak lr 2×10^{-3}, wd 0.05)	89.40 %	−0.40
no shortcut	plain network (473,450 params)	89.54 %	−0.26
no label smoothing	plain cross-entropy	90.22 %	+0.42
no cutout	flip+crop only	**90.78 %**	**+0.98**

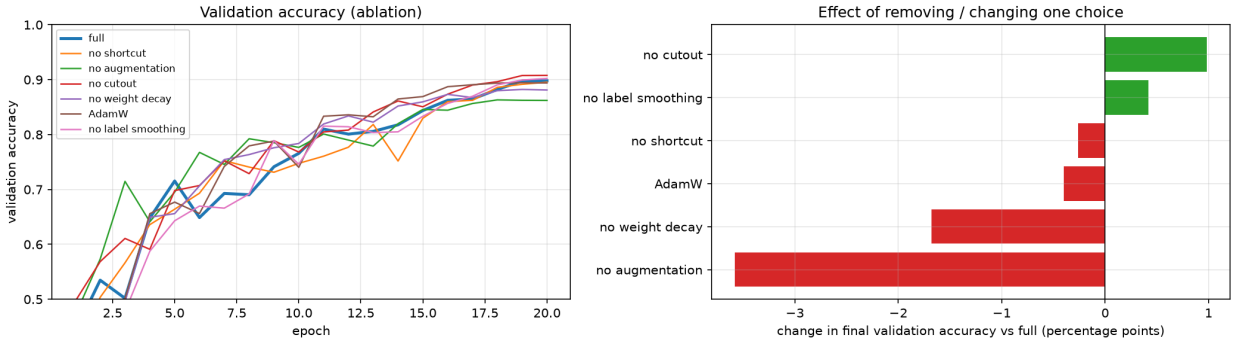


Figure 8: Validation accuracy per epoch for each variant (left), and the change in final validation accuracy relative to the full recipe (right).

Final model and test result. The selected configuration is the full recipe *without cutout*: 481,962 parameters, 20 epochs, 90.78 % validation accuracy. Its single test-set evaluation gives

$$\boxed{\text{Test accuracy} = 91.23\,\%}$$

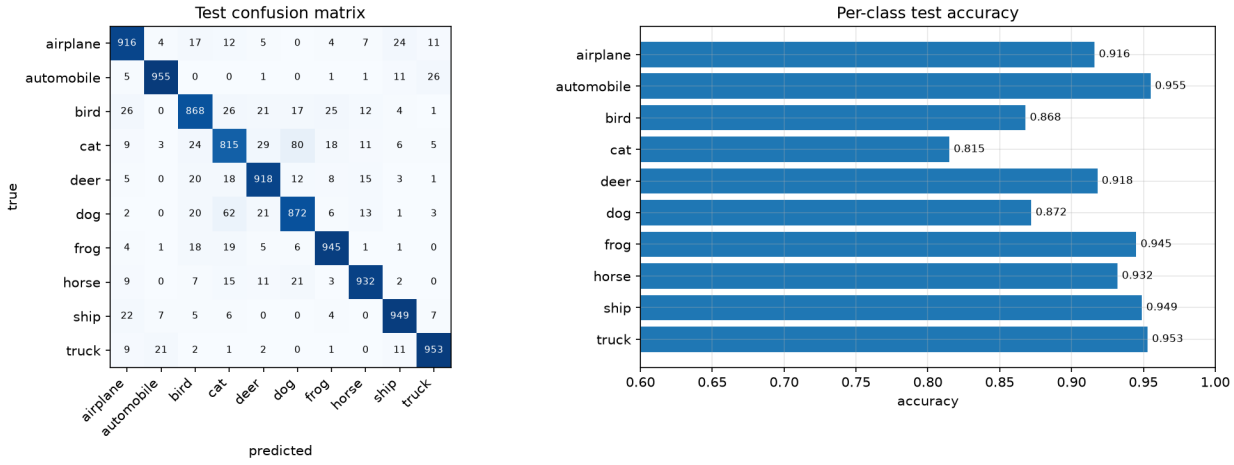


Figure 9: Test-set confusion matrix and per-class accuracy of the final model.

Interpretation.

- **Augmentation** is the most important choice (−3.58 points without it). Without it the training accuracy reaches 100 % by epoch 20, a clear overfit.

- **Weight decay** is second (-1.68). With batch norm, it keeps the weight norms small and so the effective learning rate high, which helps both optimization and generalization.
- **AdamW** is competitive but slightly below well-tuned SGD-Nesterov with one-cycle, a common result for CNNs on CIFAR.
- **Shortcuts** have only a small effect at 14 layers, consistent with 3(b), where they mattered little at depth 8–20 and a lot at depth 56.
- **Cutout and label smoothing hurt in this budget.**
  - With cutout, training accuracy is only 87.9 %, against 95.3 % without it.
  - Both are regularizers that pay off in long schedules. With 20 epochs the model is limited by under-training, not overfitting: the same bias–variance trade-off as 3(a)'s "all combined".
- **Test vs. validation:** the test accuracy (91.23 %) is within half a point of the validation accuracy (90.78 %), which is within the sampling noise of a 5k validation set. Model selection did not overfit the validation split.
- **Errors:**
  - Cat is the hardest class (81.5 %), mostly confused with dog (80 cats predicted as dogs, 62 dogs as cats).
  - Other confusions are automobile $\leftrightarrow$ truck and airplane $\leftrightarrow$ ship.
  - Vehicles are easiest: automobile 95.5 %, truck 95.3 %, ship 94.9 %.

Conclusion

- **Backpropagation:** the from-scratch layers are verified to 10^{-17} (autograd) and 10^{-9} (finite differences). Sigmoid gradients vanish by about 10^5 over 10 layers, while ReLU with He initialization keeps them balanced.
- **Optimizers:** the custom SGD-momentum and Adam match PyTorch exactly. Momentum and Adam fix ill-conditioning in different ways, and each optimizer works only within about one decade of learning rate.
- **Regularization:** with 5k images, geometric augmentation is the strongest regularizer, and stacking every regularizer underfits.
- **Residual connections:** plain-56 suffers from the degradation problem, while residual-56 is the best network. Shortcuts make depth usable.
- **Best CNN:** a 482k-parameter ResNet-14 trained for 20 epochs reaches **91.23 % test accuracy**. The ablation shows that augmentation and weight decay matter most.